

✦ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



How Razorpay Built an AI Analyst That Replaced 20 Data Scientists' Worth of Work

Inside DataGaaru — the reasoning engine quietly handling 1,500+ queries across 40 business units

7 min read · 6 days ago



Rohit Kushwaha

Follow



Listen



Share



More

Every Monday morning, the same fire drill played out at Razorpay.

A product manager needed a chargeback ratio. A risk lead wanted to see UPI transaction trends. An engineering head needed TAT numbers before a 9 AM review. All roads led to the same place: a 30-person analytics team that was drowning.

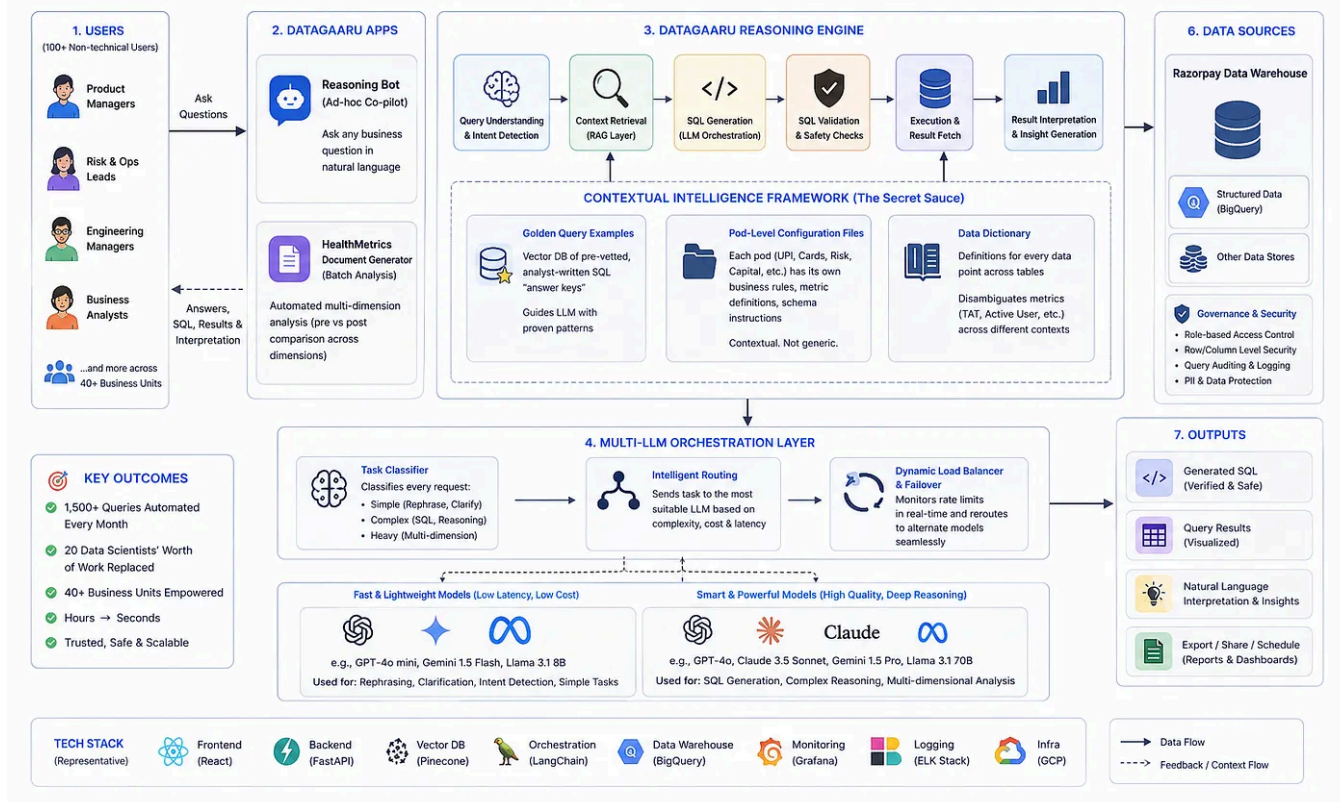
The analysts weren't slow. They weren't under-skilled. They were just *the only ones who could open the door* — and everyone was knocking at once.

That bottleneck had a name inside Razorpay: the **Monday Morning Panic**. And it birthed one of the most architecturally sophisticated internal AI tools built by an Indian tech company.

They called it **DataGaaru**.

DataGaaru: Razorpay's Self-Serve AI Reasoning Engine

From natural language to trusted insights – at scale, for everyone.



Inside DataGaaru — the reasoning engine quietly handling 1,500+ queries across 40 business units

The Problem Wasn't People. It Was the Model.

Before we get to the solution, it's worth sitting with the problem properly.

Razorpay is a full-stack fintech processing billions of dollars in payments. It has 40+ distinct business and product pods — UPI, Cards, Risk, Payroll, Capital, and more. Each pod generates data. Each pod needs answers. And every answer required a human analyst to:

1. Discover the right table across a sprawling data warehouse
2. Write the SQL (with the right joins, filters, partition logic)
3. Debug it (because the first query is almost never the right one)
4. Run it against the database
5. Interpret the output and surface an insight

That lifecycle — from question to answer — could take **hours**. Sometimes **days**.

And the real problem wasn't the volume. It was the nature of the dependency. Non-technical teams had zero autonomy. Every time a PM wanted a number, an analyst had to context-switch. Every ad-hoc data request pulled someone away from higher-value strategic work.

Scaling a human-centered analytics model was simply not going to keep pace with Razorpay's growth. Something had to break. Razorpay decided to break it on their own terms.

Meet DataGaaru: Not Just a Chatbot, a Reasoning Engine

The name "DataGaaru" (roughly translating to "Data Sir" or "Data Expert" in Telugu) signals the ambition: this isn't a simple SQL generator with a chat interface. It's a system designed to *think* — to reason about business questions and produce accurate, safe, production-grade answers.

DataGaaru splits into two specialized applications, each targeting a different use pattern:

1. The Reasoning Bot (Ad-hoc Co-pilot) Type a plain English question. Get SQL. Get a result. Get an interpretation. No analyst required. This is the self-serve engine for the constant stream of one-off data questions that were clogging the analytics team's queue.

2. The HealthMetrics Document Generator (Batch Analysis) This runs in the background for structured business reviews. Instead of a single question, it executes *parallel multi-dimension analysis* — comparing "pre-period" vs. "post-period" metrics like TAT and chargeback-to-sales ratios across multiple dimensions simultaneously. What used to take an analyst an afternoon gets generated automatically.

The architecture behind both is where it gets genuinely impressive.

The Secret Sauce: A Contextual Layer That Prevents Hallucinations

The most common failure mode for LLM-based data tools is the same everywhere: the model hallucinates a column name, joins the wrong tables, or generates SQL that runs but returns nonsense.

Razorpay's engineering team went deep on solving this. Instead of just handing an LLM a table schema and hoping for the best, they built a multi-layer **contextual intelligence framework** on top of RAG (Retrieval-Augmented Generation).

Here's how it works:

Golden Query Examples A vector database stores pre-vetted SQL queries — “answer keys” written and validated by senior analysts. When a user asks a question, the system performs semantic search to find the most relevant Golden Query and uses it to *guide* the LLM's code generation. The model isn't starting from scratch; it's referencing proven patterns.

Pod-Level Configuration Files Each business unit — UPI, Cards, Risk, Capital — has its own configuration file containing that pod's specific business rules, metric definitions, and schema instructions. When the Reasoning Bot handles a question from the Risk team, it loads the Risk pod's config. This isn't generic; it's contextually aware.

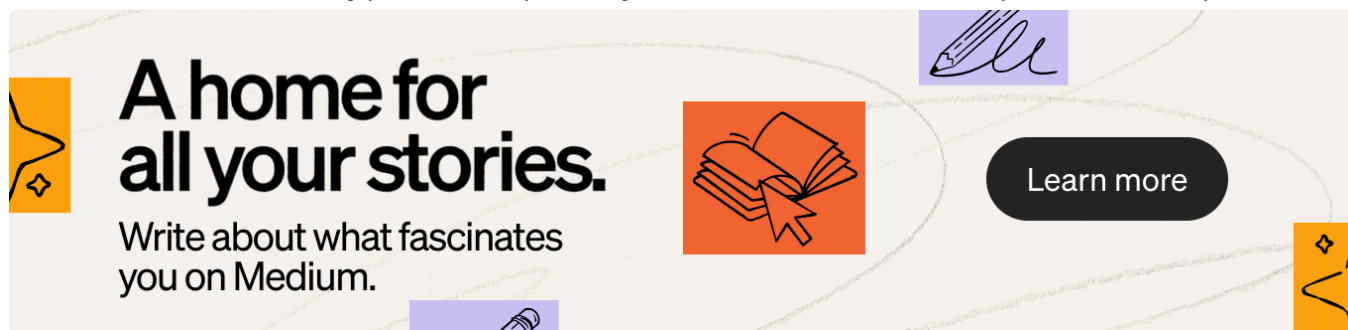
Data Dictionary Every data point across every table has a definition. The system knows what “TAT” means in the Lending context vs. the Payments context. It knows how “active user” is defined for one pod vs. another. Ambiguity is the enemy of accurate SQL, and the data dictionary kills ambiguity.

This combination — Golden Queries + pod configs + data dictionary — is what separates DataGaaru from a simple “text to SQL” wrapper. It's the difference between a model that *sometimes* gets things right and one that's reliable enough to put in front of 100+ non-technical users.

Multi-LLM Orchestration: Why One Model Isn't Enough

Here's something most internal AI tools get wrong: they pick one model and use it for everything. That's expensive, slow, and often overkill.

DataGaaru uses an intelligent routing layer that classifies every task and sends it to the right model.



Fast vs. Smart routing:

- Simple tasks like *rephrasing a natural language question* get routed to fast, lightweight models (think GPT-4o mini class). Low latency, low cost.
- Complex tasks like *SQL generation and deep business reasoning* go to high-parameter, frontier models. Worth the wait and the cost.

Dynamic load balancing: The system monitors rate limits in real-time. If one LLM is throttled, the request automatically reroutes to an alternative model. No user ever sees a 429 error or a frozen spinner because the model hit its rate limit.

This is proper production-grade thinking. It's not sexy, but it's what makes the difference between a demo that works in a presentation and a tool that 100+ people can depend on daily.

The Infrastructure: Trino, Pino, and the Concurrency Problem

Great SQL generation means nothing if query execution takes four minutes.

Razorpay runs queries against **Trino** and **Pino** database infrastructures. When they scaled the Pino infrastructure specifically, they cut query execution time from **4 minutes down to 1 minute** — while supporting over 100 concurrent users firing queries simultaneously.

That's the kind of engineering work that doesn't make a flashy demo slide but absolutely determines whether people actually use the tool every day.

Post-execution, the system doesn't just dump a raw table. **Python scripts automatically generate visualizations and first-level insights** from the returned data. The user doesn't get a 50-row result set and a good luck wish. They get a chart and a sentence that says what it means.

Security and Governance: Production-Grade from Day One

Razorpay is a regulated fintech. It handles payment data for lakhs of businesses. “Move fast and break things” is not a philosophy that survives contact with RBI compliance requirements.

DataGaaru was built with this reality baked in from the start:

Database-Level RBAC Here's the elegant part: DataGaaru doesn't maintain a parallel permissions system. It inherits the existing Role-Based Access Control from the underlying database. If a user doesn't have permission to query the payroll table in the primary database, DataGaaru won't let them query it either. The AI layer respects the organizational permission model automatically.

Input Guardrails Before any prompt reaches an LLM, a security layer runs. It redacts PII (Personally Identifiable Information) and scans for prompt injection attempts. Users can't trick the system into revealing data they shouldn't see, and sensitive information doesn't flow into third-party model APIs.

SQL Optimization Constraints The system injects mandatory instructions into every LLM prompt: use partition filters, never use `SELECT *`, enforce row limits. Not only does this prevent accidental full-table scans that would crater the database, it also keeps compute costs sane. There's even a secondary LLM pass specifically for SQL optimization before execution — the query gets reviewed before it runs.

The Feedback Loop: How DataGaaru Gets Smarter

This is the part that makes DataGaaru a *learning system* rather than a static tool.

Every query gets a thumbs-up or thumbs-down from the user. When a query earns positive feedback, it gets logged back into the vector database — it becomes a new Golden Query. The system's contextual knowledge base grows with every successful interaction.

On the human side: configuration files and data dictionary entries are initially *generated* by LLMs, but they're **manually refined by analysts** before being used in production.

This isn't pure AI-automation. It's AI doing the heavy lifting and humans providing the final 10% accuracy that takes something from "mostly right" to "always right."

It's an honest and mature approach to AI deployment — one that acknowledges where models still need a human check, rather than overselling full autonomy.

The Numbers: What This Actually Delivered

ROI on internal AI tools is notoriously hard to measure. DataGaaru's numbers are unusually concrete:

- **20% of the analytics team's bandwidth reclaimed.** In a 100-person team, that's effectively 20 analysts freed from data-pulling to do strategic work.
- **80% of company metrics covered** across all 40 business and product pods.
- **100+ active users firing over 1,500 queries** — from PMs, business leads, and engineers who previously had no self-serve data access.
- Query execution time cut from **4 minutes to 1 minute** under concurrent load.

And the qualitative shift is arguably more important than the quantitative one. Analysts at Razorpay are no longer "data pullers." They're *strategic movement partners* — the people who tell you what the data *means*, not just the people who go fetch it.

What DataGaaru Teaches Us About Building AI That Works

Most internal AI tools fail for one of three reasons: they hallucinate too much to trust, they're too slow to bother with, or they don't respect existing governance structures so IT shuts them down.

DataGaaru avoids all three failure modes through deliberate engineering choices:

- It fights hallucination with *multiple layers of context* — Golden Queries, pod configs, and a data dictionary — not just prompting tricks.
- It fights latency with *smart LLM routing* and proper database scaling.

- It fights governance risk by *inheriting* existing RBAC rather than trying to replicate it.

The deeper lesson: the hard part of building AI products isn't writing the prompt. It's everything around the prompt — the data infrastructure, the security layer, the feedback mechanisms, the operational considerations that determine whether something survives contact with real users in a production environment.

Razorpay built DataGaaru to last. That's why it works.

The technical details in this article are based on Razorpay's public breakdown of the DataGaaru architecture, shared in their engineering sessions.

[Artificial Intelligence](#)[AI Agent](#)[Agentic Rag](#)[Machine Learning](#)[System Design Interview](#)[Follow](#)

Written by Rohit Kushwaha

25 followers · 54 following

AI x Web Engineer @ W3villa | Python, Ruby on Rails, React | Building GenAI Products with Agentic Workflows & RAG Systems

No responses yet



shubham badaya

What are your thoughts?

